

Experiment 2: Case Study on SDLC and Threat Modelling

Aim: To study the login module of an application, identify five vulnerabilities and the threats they create, state which security goals (CIA) are at risk, and explain which design mechanism controls each one.

Introduction to SDLC and Threat Modelling

The Software Development Life Cycle (SDLC) is the ordered set of phases a team follows to plan, build, test, release and maintain software. Security is cheapest and most effective when it is addressed in every phase instead of being added at the end:

- **Requirements:** decide what must be protected and which standards apply (for example, MFA for all accounts).
- **Design:** choose a secure architecture and run threat modelling before any code is written.
- **Implementation:** follow secure coding rules, use vetted libraries and keep secrets out of source code.
- **Testing:** run static analysis, dependency scans and penetration tests against abuse cases.
- **Deployment:** harden configuration, enforce HTTPS and rotate keys.
- **Maintenance:** monitor logs, patch quickly and respond to incidents.

Threat modelling is a structured way of finding what can go wrong with a system, ranking the risks and deciding on controls. The STRIDE model groups threats into six types:

Threat type	Description	Login-module example
Spoofing	Pretending to be another user or system	Logging in with stolen credentials
Tampering	Modifying data or messages	Altering a session token or redirect parameter
Repudiation	Denying an action took place	Claiming a sign-in was not made by the user
Information disclosure	Exposing data to the wrong party	Password or token leakage
Denial of service	Making the system unavailable	Flooding the login endpoint
Elevation of privilege	Gaining rights not granted	Using a normal session to reach admin actions

Security Goals: the CIA Triad

- **Confidentiality:** information is available only to authorized users.
- **Integrity:** data and actions cannot be altered without detection or authorization.
- **Availability:** the service is usable by legitimate users when they need it.

Case Study: GitHub Login Module (Password + 2FA, OAuth and Sessions)

System context: GitHub lets users sign in with a username or email and password, and it expects a second factor (authenticator app, SMS, security key or passkey). Sign-in from an unrecognised device triggers an extra verification step. Users can also sign in to third-party applications through GitHub's OAuth flow, and an active login is kept alive with a browser session cookie. This case study looks at that sign-in and session flow.

Vulnerability, Threat and Mitigation Analysis

No.	Identified vulnerability	Associated threat	Security goals at risk	Design control / mechanism
1	Reused or weak passwords Users choose guessable passwords or reuse ones leaked from other sites.	Credential stuffing and password guessing Attackers replay leaked username and password lists against the login form.	Confidentiality, Integrity	Mandatory 2FA and breached-password checks: a password alone is not enough to sign in; a second factor (TOTP, security key or passkey) is required, and passwords found in known breaches are rejected.
2	Unlimited login attempts No limit on how many times credentials can be tried.	Brute force and denial of service Automated tools hammer the endpoint to guess passwords or lock out real users.	Availability, Confidentiality	Rate limiting and bot detection: throttling per IP and per account, progressive delays and CAPTCHA challenges after repeated failures.
3	Cross-site request forgery (CSRF) State-changing requests are accepted without proof they came from the site's own pages.	Session riding and forged actions A malicious page makes a signed-in user's browser perform actions on their account.	Integrity	Anti-CSRF tokens and SameSite cookies: every form carries an unpredictable token checked on the server, and session cookies use SameSite so they are not sent on cross-site requests. OAuth flows add a random <i>state</i> value.
4	Insecure session cookie handling Session identifiers readable by scripts or sent over plain HTTP.	Session hijacking via XSS or network sniffing An attacker steals the session cookie and acts as the user without a password.	Confidentiality, Integrity	HttpOnly, Secure cookies and HTTPS everywhere: cookies cannot be read by JavaScript and travel only over TLS (with HSTS); users can review and revoke active sessions from their settings.
5	Phishing of credentials and codes Users can be tricked into typing credentials on a look-alike site.	Account takeover through spoofed login pages A fake page captures the password and relays one-time codes in real time.	Confidentiality, Integrity	Phishing-resistant authentication and device verification: passkeys and security keys are bound to the real domain so they do not work on a fake site; sign-ins from new devices trigger an extra verification and a notification email.

Additional protection: sensitive actions such as changing the password, adding a deploy key or deleting a repository ask the user to confirm again (GitHub's *sudo mode*), which limits damage if a session is hijacked.

Analysis of Security Goals

Goal	How it is at risk in this login module	Control that protects it
Confidentiality	Stolen passwords, session cookies and one-time codes expose private repositories.	2FA, passkeys, HttpOnly/Secure cookies, HTTPS

Goal	How it is at risk in this login module	Control that protects it
Integrity	A hijacked session or forged request can change code, settings or collaborators.	CSRF tokens, SameSite cookies, sudo mode
Availability	Login flooding or lockout abuse can prevent legitimate sign-ins.	Rate limiting, bot detection, progressive delays

Screenshots

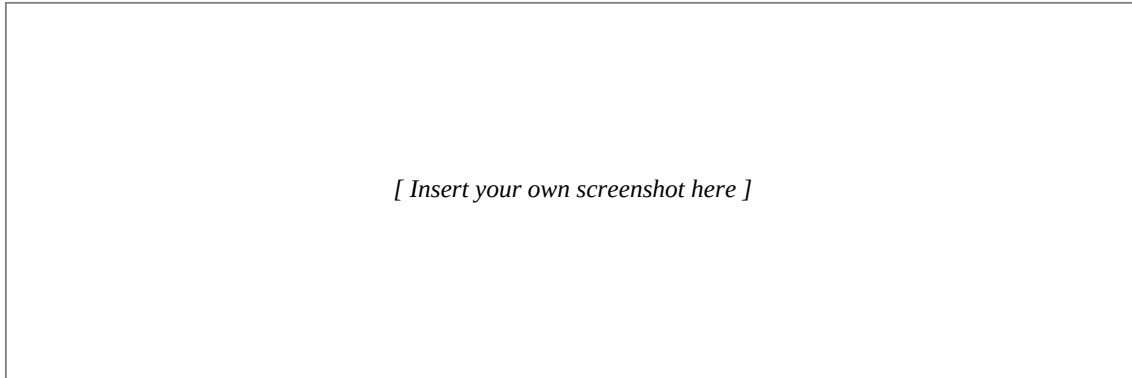


Figure 1: GitHub sign-in page (username and password step).

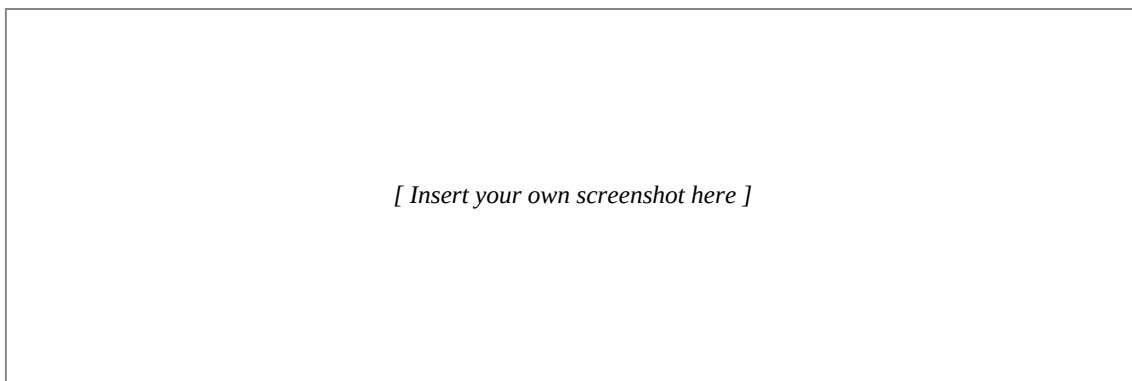


Figure 2: Two-factor authentication prompt after the password is accepted.

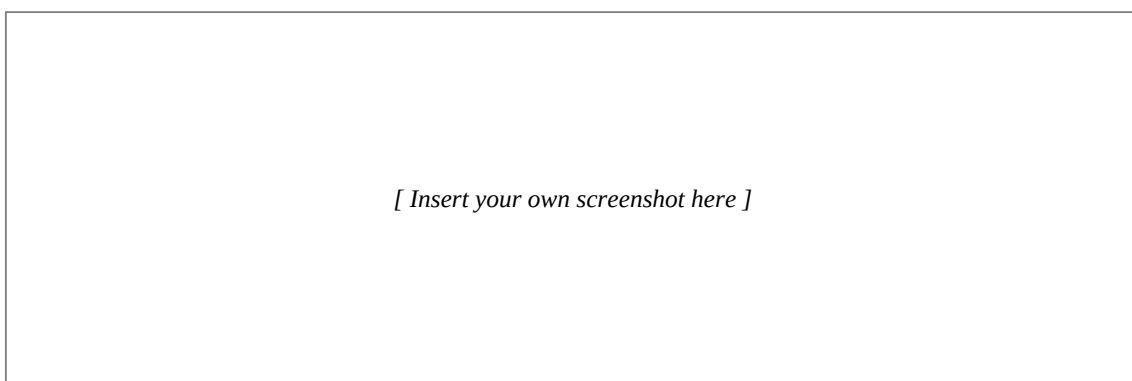


Figure 3: Settings, Sessions page listing active devices with the option to revoke them.

Conclusion

Threat modelling the GitHub login module with STRIDE shows that most risks come from the credential, the second factor and the session that follows a successful login. Each identified vulnerability is covered by a specific design mechanism: 2FA and breach checks against stolen passwords, rate limiting against brute force, CSRF tokens and SameSite cookies against forged requests, HttpOnly/Secure cookies against

session theft, and phishing-resistant passkeys with device verification against fake login pages. Applying these controls from the design phase of the SDLC, instead of adding them after release, keeps all three CIA goals protected.